THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE
CS-4031 Compiler Construction — Fall 2026

Lecture 3

Lexical Analysis: Tokens, Patterns, and Lexemes; the Chomsky Hierarchy; Regular Languages

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§3.1 (The Role of the Lexical Analyzer), §3.3 (Specification of Tokens: terminology, strings, languages, operations)
Other references:	[AP] Modern Compiler Implementation, Ch. 2

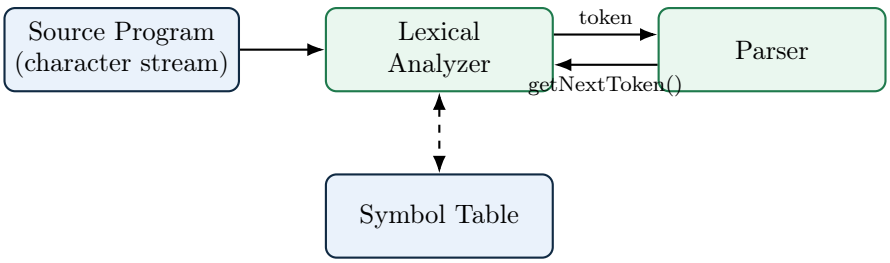
Lecture Roadmap

1. Where lexical analysis sits, and exactly how it talks to the parser; why lexer and parser are separate phases.
2. The three core vocabulary words of this unit: **token**, **pattern**, **lexeme** – with several fully worked examples, plus token **attributes** and **lexical errors**, including a close look at **panic-mode recovery**.
3. **The big picture**: how a regular expression eventually becomes a working, table-driven lexer, via finite automata – the roadmap for this lecture and the ones after it.
4. The **Chomsky hierarchy**: how formal, regular, context-free, and more powerful language classes relate to each other, and which machines recognize each one.
5. **Regular languages**, defined formally and precisely – the last topic of today, and the foundation the next lecture builds **regular expressions** on top of.

By the end of this lecture you will know exactly which languages qualify as "regular" and why, and you will have the full roadmap for how such a language eventually becomes a running lexer. Lecture **??** picks up immediately from here: regular expressions, the notation we use to *write down* a regular language.

3.1 Where Lexical Analysis Fits

[DB] The **lexical analyzer** (also called a **scanner** or **lexer**) is the first phase of a compiler. It reads the raw stream of characters making up the source program and groups them into a stream of **tokens** — meaningful chunks like keywords, identifiers, numbers, and operators — that the rest of the compiler works with instead of raw text.



How the Lexer and Parser Actually Talk to Each Other

In nearly every real compiler, the lexer does *not* tokenize the whole file up front and hand the parser a list. Instead, the parser drives the process: whenever the parser needs another token, it calls a function conventionally named `getNextToken()`. The lexer resumes scanning exactly where it left off, produces the next single token, and returns it. This "pull" model, sometimes called treating the lexer as a **coroutine** of the parser, avoids ever materializing the full token list in memory and lets the lexer and parser interleave their work one token at a time.

3.1.1 Why Not Just One Phase?

[DB] DB gives three concrete engineering reasons for splitting lexical analysis out as its own phase rather than folding it into the parser:

1. **Simplicity of design.** Separating out low-level tasks (recognizing keywords, numbers, string literals) lets the parser work purely with grammatical structure, using tokens as its atomic symbols, instead of also worrying about individual characters.
2. **Improved efficiency.** Specialized, table-driven techniques for recognizing tokens – built from finite automata, which the lectures right after this one construct *directly* from the regular expressions we develop later in this very lecture – are considerably faster than the general-purpose parsing techniques would be if forced to operate character-by-character.
3. **Enhanced portability.** Input-device and operating-system-specific quirks — end-of-line conventions (`\n` vs. `\r\n`), character encodings, special end-of-file characters — are confined entirely to the lexer. The parser, and everything after it, is completely insulated from these differences.

Exam Focus

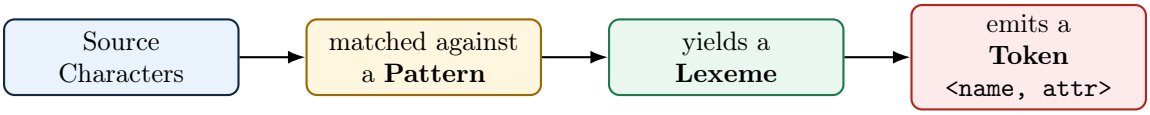
This "three reasons" list (simplicity, efficiency, portability) is a favorite short-answer question. Be able to name and briefly justify all three, not just one.

3.2 Tokens, Patterns, and Lexemes

[DB] These three terms are used precisely and are easy to blur together. Getting the distinction exactly right is one of the most exam-tested ideas in this entire unit.

Definition: Token, Pattern, Lexeme

- A **token** is a pair `<token-name, attribute-value>` representing a *category* of lexical unit, e.g. `ID`, `NUM`, `RELOP`. The token-name is an abstract symbol used by the parser.
- A **pattern** is the rule that describes the *set of possible lexemes* that can produce a given token – for identifiers, informally, "a letter followed by any number of letters and digits." Lecture **??** makes patterns precise using regular expressions.
- A **lexeme** is the actual sequence of characters in the source program that *matches* a pattern and is classified as an instance of some token.



Example: DB-Style Token/Pattern/Lexeme Table

Token	Informal Description of Pattern	Sample Lexemes
`WHILE`	the exact keyword `while`	`while`
`ID`	a letter, followed by any number of letters or digits	`i`, `sum`, `score2`
`NUM`	any numeric constant	`0`, `42`, `3.14`
`RELOP`	<, <=, =, <>, >, or >=	<=, <>
`LITERAL`	any characters between a pair of double quotes, except a double quote itself	`"Total = "`

3.2.1 Worked Example 1: Tokenizing a Function Call

Consider the following line of C-like source code:

```
printf ( "Total = %d\n" , score ) ;
```

identifier punctuation/operator string literal

Lexeme	Token Name	Attribute-Value (passed to parser)
`printf`	`ID`	pointer to symbol-table entry for `printf`
`(`	`LPAREN`	none needed
`"Total = %d\n"`	`LITERAL`	pointer to the string's stored value
`,`	`COMMA`	none needed
`score`	`ID`	pointer to symbol-table entry for `score`
`)`	`RPAREN`	none needed
`;`	`SEMI`	none needed

3.3 Attributes for Tokens

[DB] When more than one lexeme can match a token's pattern (as with `ID` or `NUM`), the lexer must pass along enough extra information – the **attribute-value** – for later phases to know exactly *which* identifier or *which* number was matched. For identifiers, this is essentially always a pointer into the symbol table.

Example: Worked Example 2: An Assignment Statement

Source line: `E = M * C ** 2`

Assume `**` is this language's exponentiation operator. The lexer's output, as a stream of `<token-name, attribute-value>` pairs, is:

```
<ID, ptr to symtab entry for E>
<ASSIGN_OP, - >
<ID, ptr to symtab entry for M>
<MULT_OP, - >
<ID, ptr to symtab entry for C>
<EXP_OP, - >
<NUM, integer value 2>
```

Notice: `ID` tokens all carry a symbol-table pointer as their attribute so that later phases can distinguish `E` from `M` from `C`; the operator tokens need no attribute at all, since the token-name

alone (`ASSIGN_OP` vs. `MULT_OP`) already says everything the parser needs to know.

Exam Focus

Given a short program line, be able to produce the exact stream of `<token-name, attribute-value>` pairs, in order, the way both worked examples above do. This is the single most common long-answer question style for this unit.

3.3.1 Worked Example 3: A Complete Statement Block

```
int sum = 0;
while (i <= 10) {
sum = sum + i;
i = i + 1;
}
```

Lexeme	Token	Attribute	Lexeme	Token	Attribute
int	TYPE	–	sum	ID	ptr(sum)
sum	ID	ptr(sum)	=	ASSIGN	–
=	ASSIGN	–	sum	ID	ptr(sum)
0	NUM	0	+	PLUS	–
;	SEMI	–	i	ID	ptr(i)
while	WHILE	–	;	SEMI	–
(	LPAREN	–	i	ID	ptr(i)
i	ID	ptr(i)	=	ASSIGN	–
<=	RELOP	LE	i	ID	ptr(i)
10	NUM	10	+	PLUS	–
)	RPAREN	–	1	NUM	1
{	LBRACE	–	;	SEMI	–
			}	RBRACE	–

Two details worth noticing in this longer example:

- `sum` and `i` each get *their own* symbol-table pointer the first time they're seen, and the *same* pointer every time they reappear – the symbol table guarantees one entry per distinct identifier, no matter how many times it's used.
- Whitespace (the spaces and newlines between tokens) and indentation produce *no* tokens at all – they are consumed and discarded by the lexer, along with any comments, before the parser ever sees anything.

The Lexer's "Housekeeping" Duties

Beyond producing tokens, DB notes the lexer is also the natural place to: strip out whitespace and comments (neither produces a token); and track line numbers (and often column numbers) so that later phases can report errors with accurate source locations, e.g. "`undeclared variable 'x' at line 42`."

3.4 Lexical Errors

[DB] It is surprisingly rare for a lexer to be able to detect an error entirely on its own, *by itself, in isolation* – because almost any short character sequence is a *prefix* of some valid lexeme for some token.

Example: The Classic Illustration

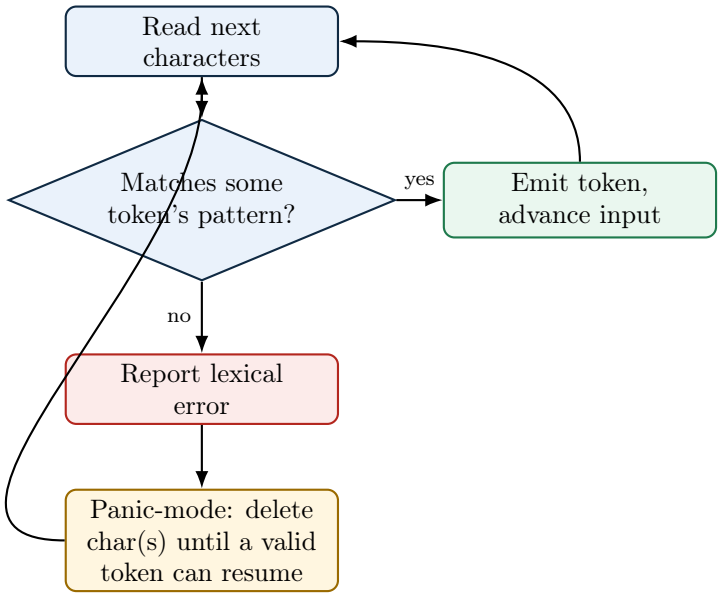
Consider the fragment `fi (a == f(x))` in a C-like language, where the programmer *meant* to type the keyword `if`. The lexer cannot know this. `fi` is a perfectly legal lexeme for token `ID` ("a letter followed by letters/digits"), so the lexer happily emits `<ID, ptr(fi)>`. The mistake will only be caught later – by the *parser*, if `fi` appears somewhere no identifier is grammatically valid, or by *semantic analysis*, if `fi` is used as if it were a function without ever having been declared.

Genuine lexical errors are ones where *no pattern for any token* can match the remaining input at all, or where a token's own internal structure is malformed:

- An illegal character that starts no valid token in the language, e.g. a stray `#` or `@` in a language that assigns those characters no meaning.
- An unterminated string literal: opening quote seen, but the line (or file) ends before a matching closing quote.
- A malformed numeric literal, e.g. `3.14.15` (two decimal points), or `2r` immediately followed by a letter with no valid meaning in that position.
- An unterminated comment: a `/*` seen with no matching `*/` anywhere before end of file.

3.4.1 Error Recovery Strategies

[DB] A production-quality lexer does not simply stop at the first error — it tries to recover and keep going, so the programmer can see *several* distinct errors from one compilation attempt rather than fixing them one at a time.



[DB] Panic mode, in detail. "Panic mode" names a very specific, very simple procedure, and it helps to spell out its steps exactly, since the name alone makes it sound more dramatic than it is: (1) report the error at the current position; (2) delete (discard) the current character; (3) check whether the *remaining* input now begins with something matching a valid token's pattern; (4) if not, go back to step 2 and delete the next character too; (5) as soon as the remaining input does match a valid token – or a natural boundary like end of line or end of file is reached – stop deleting and resume ordinary scanning from there. In short: *throw away characters, one at a time, until scanning can safely restart.*

Example: Worked Example 1: A Single Illegal Character

Source line: `total = x @ y;`
The lexer scans `total` (`ID`), `=` (`ASSIGN`), `x` (`ID`) normally. It then reaches `@`, which matches no token pattern in this language. **Step 1:** report "illegal character `@` at column 12." **Steps 2–3:** delete just the `@`; the very next character, a space, already leads into a lexeme (`y`) that matches `ID`. So panic mode stops after deleting a single character. Scanning resumes normally: `y` (`ID`), `;` (`SEMI`). One error reported, and the rest of the line recovers cleanly – this is the best case for panic mode.

Example: Worked Example 2: Discarding Until a Natural Boundary

Source:

```
printf("Hello world);
x = 5;
```

The lexer scans `printf` (`ID`), `(` (`LPAREN`), then starts matching a `LITERAL` token at the opening `"`. It looks for a matching closing `"` but reaches the end of the line without finding one. Report: "unterminated string literal starting at line 1." Unlike Worked Example 1, there is no small number of characters to delete that fixes this – *nothing* inside a broken string is a safe resumption point. Panic mode instead discards everything from the opening quote through the end of the line (a natural boundary most languages use for exactly this reason: string literals cannot normally span a raw newline), and resumes scanning fresh at line 2: `x` (`ID`), `=` (`ASSIGN`), `5` (`NUM`), `;` (`SEMI`). One error reported; the rest of the program is still checked for further problems.

Example: Worked Example 3: Why Panic Mode Can Cascade Into Extra Errors

Source line: `y = 3.14.159 + 2;`
The lexer scans `y` (`ID`), `=` (`ASSIGN`), then starts matching a `NUM` at `3.14`. Maximal munch tries to extend the match, but a *second* `.` cannot extend a valid `num` (DB's regular definition, covered fully with Lecture **??**'s `num` example, allows only one optional fraction part) – so the lexer accepts what it *did* successfully match, emits `<NUM, 3.14>`, and resumes scanning from the very next character: a bare `..` But a lone `.` does not start any valid token in this language either, so panic mode fires a *second* time, reporting a second illegal-character error and deleting just that `.`, before finally matching `159` as another `NUM` token. **One root mistake – a stray decimal point – surfaces as two reported errors.** This is a known, unavoidable downside of panic mode: it recovers *syntactically* (scanning can always continue), not *semantically* (it has no idea what the programmer actually meant), so one typo can occasionally produce a short burst of related-looking error messages. Experienced programmers learn to fix the *first* error in a cluster and recompile, rather than trying to fix every reported line individually.

[DB] Beyond panic mode: local corrections. DB also lists four more surgical "local-correction" strategies a recovering lexer *could* attempt instead of blindly discarding characters – though, as the note after the examples explains, only two of them apply cleanly to genuine lexical errors:

- **Deleting** an extraneous character – e.g. the malformed number `3..14` (one `.` too many): deleting the second `.` recovers the valid number `3.14`, instead of panic mode's blunter "discard until something matches."
- **Inserting** a missing character – e.g. the unterminated string `"Hello` with the line ending before a closing quote: inserting a `"` right where the line ends recovers a (short) valid string literal, letting scanning continue past it immediately instead of discarding the rest of the line as Worked Example 2 did.
- **Replacing** one character with another – e.g. `3@4` plausibly intended as `3*4`: replacing `@` with `*` recovers a valid expression.

- **Transposing** two adjacent characters – e.g. `"Helol"` plausibly intended as `"Hello"`.

Exam Focus

Not all four local corrections are equally "lexical." **Deletion** and **insertion** directly repair the two genuine categories of lexical error from §3.4 – an extra character the pattern can't accommodate, or a character the pattern is missing – so a lexer's own panic-mode-plus-local-correction logic can apply them without knowing anything about programmer intent. **Replacement** and **transposition**, by contrast, mostly turn one *valid* lexeme into a *different* valid lexeme (as the `Helol`→`Hello` example shows) – exactly the "looks like a typo but is a perfectly legal token" situation from the `fi`/`if` example earlier in this lecture, which is *not* a lexical error at all and is therefore outside what the lexer itself can or should silently fix. Replacement and transposition matter far more for IDE-style "did you mean . . . ?" suggestions layered on top of the compiler than for the lexer's own error-recovery logic.

A Bit of History

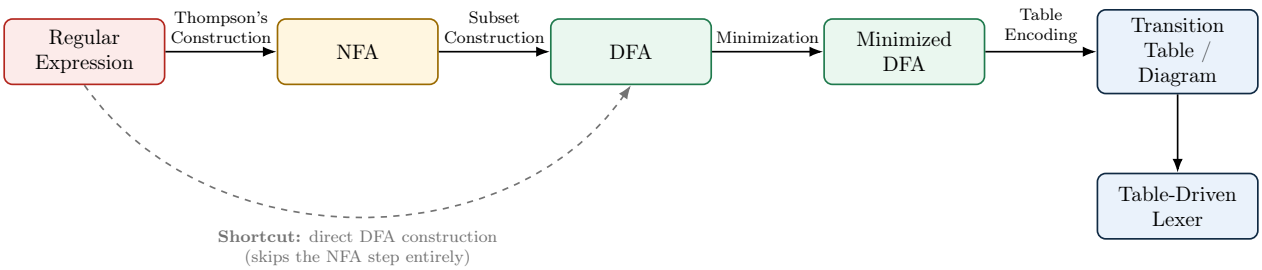
DB points toward a more principled version of this idea: choosing whichever single correction requires the *fewest* character edits – insertions, deletions, substitutions – to turn the bad input into something valid. This is exactly **edit distance** (Levenshtein distance), the same string-similarity measure used today in spell-checkers and DNA sequence alignment. Aho and Peterson described a minimum-distance error-correcting parser as early as 1972; the idea that "the best fix is the cheapest fix" turns out to generalize far beyond lexical analysis, into syntax error recovery and even runtime auto-correct systems.

Exam Focus

Be able to explain, with a concrete example like `fi (a == f(x))`, *why* a misspelled keyword is usually **not** a lexical error even though it looks like an obvious typo. Then be able to walk through panic mode step by step (report, delete, check, repeat, resume) on a short broken input the way the three worked examples above do – this is the most common long-answer version of this topic, and students often lose marks by describing panic mode too vaguely ("it skips bad characters") instead of naming the actual step-by-step procedure.

3.5 The Big Picture: From Regular Expressions to a Working Lexer

[DB] Every pattern we have used so far has been informal English: "a letter followed by letters or digits." That is fine for talking about lexical analysis, but it is not something a computer can execute. Turning informal patterns into an actual running lexer takes a short pipeline of well-defined, *mechanical* steps, and it is worth seeing the whole pipeline once, up front, before building any one piece of it.



Step	What Happens
Regular expression → NFA	**Thompson's construction**: a purely mechanical algorithm that builds a nondeterministic finite automaton (NFA) directly from the structure of a regular expression, one small automaton per operator.
NFA → DFA	**Subset construction**: another mechanical algorithm that converts any NFA into an equivalent deterministic finite automaton (DFA) – one with no ambiguity about which transition to take, at the cost of (possibly) more states.
(shortcut) Regular expression → DFA directly	Some tools skip the NFA step entirely: DB §3.9 builds a DFA *directly* from a regular expression's syntax tree, using functions traditionally called *nullable*, *firstpos*, *lastpos*, and *followpos*. It computes exactly the same DFA that Thompson's construction plus subset construction would have produced (possibly needing a separate minimization pass afterward) – it is a different *route* to the same destination, not a different destination, and it is what many real lexer generators use internally because it tends to build fewer intermediate states.
DFA → minimized DFA	**DFA minimization**: merges states that are behaviorally indistinguishable, producing the smallest possible DFA that still recognizes exactly the same language.
Minimized DFA → transition table	The (finite!) set of states and transitions is encoded as a 2-D table: `table[state][input_symbol] = next_state`.
Transition table → lexer	A tiny, fast loop – look up the current state and next character in the table, move to the next state, repeat – *is* the lexer's inner loop. No more cleverness is needed once the table exists.

Exam Focus

"Transition diagram" and "transition table" are not two different stages of this pipeline – they are two different *representations of the same DFA (or NFA)*. A **transition diagram** is the picture: circles for states, labeled arrows for transitions, exactly the automaton drawings you will see in the next few lectures. A **transition table** is the same information laid out as a 2-D array instead of a picture. Neither one is more "correct" than the other, and neither is a separate construction step – once you have a DFA, you already have both, you have just chosen how to write it down. A common exam trap asks you to convert between the two for a small automaton; that is a notation exercise, not a new algorithm.

Why This Pipeline Matters

Every solid arrow in the diagram above is a *mechanical, automatable* algorithm – none of them require human judgment once the previous step is done, and the dashed shortcut is simply an alternative route to the same DFA. This is exactly why lexer-generator tools like Flex exist: you write only the left-most box (a set of regular expressions, one per token), and the tool runs the entire rest of the pipeline for you, emitting ready-to-compile scanner code. The remaining lectures on lexical analysis build this pipeline one arrow at a time – Thompson's construction and subset construction first, then minimization – so that by the time we reach Flex itself, you understand exactly what the tool is doing on your behalf rather than treating it as a black box.

3.6 Strings, Alphabets, and Languages

[DB] Before regular expressions can be defined precisely, three primitive terms need to be pinned down exactly as DB uses them.

Definition: Alphabet, String, Language

- An **alphabet** Σ is any finite set of symbols, e.g. $\{0, 1\}$ or the set of ASCII characters.
- A **string** (or *word*) over Σ is a finite sequence of symbols drawn from Σ. The **empty string**, written ε, has length 0.
- A **language** over Σ is simply *any* set of strings over Σ — possibly infinite, possibly empty ($\emptyset$, the language with no strings at all, which is *not* the same as $\{\varepsilon\}$, the language containing only the empty string).

Exam Focus

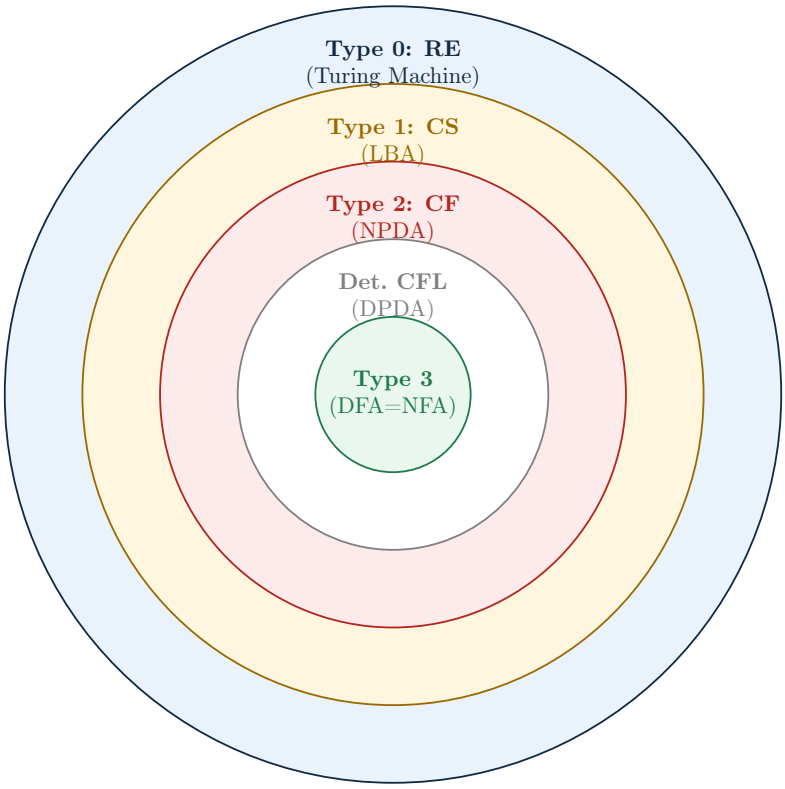
$\emptyset$ and $\{\varepsilon\}$ are a classic confusion. $\emptyset$ contains *zero* strings; $\{\varepsilon\}$ contains *one* string (the empty one). Keep them distinct.

Formal Languages: A Hierarchy, Not One Thing

The definition above – "a language is any set of strings" – deliberately puts *no* restriction on what that set looks like. A set meeting only this bare definition is often called a **formal language**, precisely to flag that nothing about its internal structure is assumed yet. Computer science studies several increasingly restrictive (and increasingly cheap to process) *classes* of formal languages, most famously organized into the **Chomsky hierarchy** – the subject of the subsection below.

3.6.1 The Chomsky Hierarchy

[DB] Linguist Noam Chomsky proposed this hierarchy in 1956 while studying the structure of natural language; computer science adopted it almost immediately, because it turns out to classify exactly the right notion for programming languages too – *how much machinery do you need to recognize this language?* Each class below is defined by a restriction on the *grammar* allowed to generate it, and each corresponds to exactly one class of recognizing machine.



RE = Recursively Enumerable, CS = Context-Sensitive, CF = Context-Free – full names and machines in the table below.

Each ring properly contains *the ones inside it: every regular language is context-free, every context-free language is context-sensitive, and so on – but not the reverse.*

Type	Class	Machine	Deterministic vs. Nondeterministic
3	Regular	DFA or NFA	**Equal power.** Every NFA has an equivalent DFA (via the *subset construction* we build toward in the next few lectures) – nondeterminism buys convenience of expression here, not extra recognizing power.
–	Deterministic CFL	DPDA	**Strictly weaker than NPDA.** Deterministic pushdown automata recognize only a proper subset of the context-free languages – this middle ring is not one of Chomsky's original four types, but is a genuinely useful refinement between Types 2 and 3.
2	Context-Free	PDA (nondeterministic, NPDA)	**Strictly stronger than DPDA.** The classic separating example: $\{ww^R : w \in \{a, b\}^*\}$ (even-length palindromes) is context-free, but no DPDA can recognize it – a deterministic machine can't "guess" where the middle of the string is without trying multiple possibilities at once.
1	Context-Sensitive	Linear-Bounded Automaton (LBA)	**Unknown!** Whether deterministic and nondeterministic LBAs have equal power is the **LBA problem**, open since 1964 – one of the oldest unsolved questions in theoretical computer science.
0	Recursively Enumerable	Turing Machine	**Equal power, and this is the ceiling.** Every reasonable model of computation ever proposed – multi-tape TMs, nondeterministic TMs, RAM machines, λ-calculus – has been proven exactly as powerful as the single-tape deterministic TM. This equivalence, and the belief that no more powerful model of *effective computation* exists, is the **Church–Turing thesis**.

Exam Focus

Keep the **NFA/DFA** case and the **PDA/DPDA** case straight – they go opposite ways, and exams like to test exactly this contrast. For *finite* automata, adding nondeterminism costs nothing in power (only in state count). For *pushdown* automata, adding nondeterminism genuinely lets you recognize more languages. This single fact is the formal reason a lexer can always be built deterministically (fast, table-driven), while a general context-free parser cannot always be – some grammars force real backtracking or lookahead machinery that a DPDA-equivalent design cannot avoid.

This lecture defines exactly one of these classes with full precision – the **regular languages**, starting below. **Context-free languages** are the parser's formalism, and the course returns to them once we reach syntax analysis; every regular language is also context-free (but not the reverse), which is exactly why a parser can express everything a lexer's regular expressions can, and more.

3.7 Regular Languages: A Formal Definition

[DB] To say precisely which languages qualify as "regular," we first need a small set of operations for building new languages out of old ones. Given two languages L and M over the same alphabet, DB defines four such operations:

Operation	Notation	Definition
Union	$L \cup M$	$\{ s \mid s \in L \text{ or } s \in M \}$
Concatenation	LM	$\{ st \mid s \in L \text{ and } t \in M \}$ (paste a string from L directly onto a string from M)
Kleene closure	L^*	$\bigcup_{i=0}^{\infty} L^i$ – zero or more strings from L, concatenated (includes ε, from L^0)
Positive closure	L^+	$\bigcup_{i=1}^{\infty} L^i$ – one or more strings from L, concatenated (excludes ε unless $\varepsilon \in L$)

Example: Worked Example: Letters and Digits

Let $L = \{a, b, c, \ldots, z, A, \ldots, Z\}$ (the 52 letters) and $D = \{0, 1, \ldots, 9\}$ (the 10 digits), each treated as a language of one-character strings.

- $L \cup D$ – every single letter *or* single digit: 62 strings, each of length 1.
- LD – every string formed by one letter followed by one digit, e.g. `a5`, `Z0` – exactly $52 \times 10 = 520$ strings, each of length 2.
- L^4 – every string of exactly 4 letters, e.g. `Test`, `abcd` – there are 52^4 such strings in total.
- L^* – every string of letters of *any* length, including ε — infinitely many strings.
- $L(L \cup D)^*$ – one letter, followed by zero or more letters-or-digits. This is *exactly* the language of identifiers in most C-like languages! We will turn this directly into a regular expression in Lecture **??**.

[DB] With these operations in hand, we can now say precisely what "regular" means – a statement about the language itself, as a set of strings, before we ever write a single regular expression on paper.

Definition: Regular Language (Inductive Definition)

The **regular languages** over an alphabet Σ are defined inductively, using exactly the union, concatenation, and Kleene-closure operations above.

Base cases. Each of the following is a regular language:

(a) $\emptyset$ – the empty language, containing no strings at all;
(b) $\{\varepsilon\}$ – the language containing only the empty string;
(c) $\{a\}$, for every symbol $a \in \Sigma$ – the language containing just that one single-character string.

Inductive cases. If L and M are regular languages, then each of the following is *also* a regular language:

(1) $L \cup M$ – their union;
(2) LM – their concatenation;
(3) L^* – the Kleene closure of L.

Closure clause. Nothing else is a regular language: the regular languages are exactly the languages reachable by finitely many applications of the three inductive cases above, starting only from the three base cases.

This definition tells us *which* languages count as regular. It does *not*, by itself, give us a convenient way to write such a language down on paper or in code – and that gap is exactly what a regular expression fills.

Regular Language vs. Regular Expression

A **regular language** is a set of strings satisfying the definition above – a semantic object, a mathematical fact about which strings belong to it. A **regular expression** (next lecture) is a piece of *syntax*: a compact, human-writable notation for naming one specific regular language. Every regular expression denotes exactly one regular language, and — a real theorem, proved properly in your Automata Theory course — every regular language can be named by at least one regular expression. The two notions therefore describe exactly the same class of languages, but they are conceptually different: one is a fact about a language, the other is a way of writing it down.

Exam Focus

"Is this a regular language?" and "is this a valid regular expression?" are different questions, and exam wording sometimes blurs them on purpose. A language can be regular even if you haven't yet written down a regular expression for it (you just haven't found the notation yet); conversely, once we define regular expressions formally next lecture, every string of regex syntax that type-checks will denote *some* regular language, whether or not that language is a useful one.

Key Takeaways

- A **lexeme** is matched source text; a **token** is the abstract `<name, attribute>` pair a lexer emits for it; a **pattern** is the rule connecting the two.
- Genuine **lexical errors** are things no pattern can match at all (illegal characters, unterminated strings/comments, malformed numbers) – a misspelled keyword is *not* one of these, since it's still a valid identifier lexeme.
- **Panic mode** reports an error, then discards characters one at a time until scanning can safely resume – simple, always terminates, but can occasionally cascade into extra reported errors from one root mistake.
- The big picture: **regular expression** $\rightarrow$ **NFA** $\rightarrow$ **DFA** $\rightarrow$ **minimized DFA** $\rightarrow$ **transition table** $\rightarrow$ **lexer**, via Thompson's construction, subset construction, and minimization (or a direct regex-to-DFA shortcut) – every step mechanical, which is exactly why lexer generators can automate all of it.
- The **Chomsky hierarchy** nests formal languages by how much machine power they need: regular (DFA = NFA) $\subset$ context-free (PDA, strictly more powerful nondeterministic than deterministic) $\subset$ context-sensitive (LBA) $\subset$ recursively enumerable (Turing machine, the most powerful model known).
- A **regular language** is a set of strings, defined inductively from three base cases ($\emptyset$, $\{\varepsilon\}$, $\{a\}$) and three inductive cases (union, concatenation, Kleene closure). A **regular expression** – next lecture – is the notation we use to name one.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.